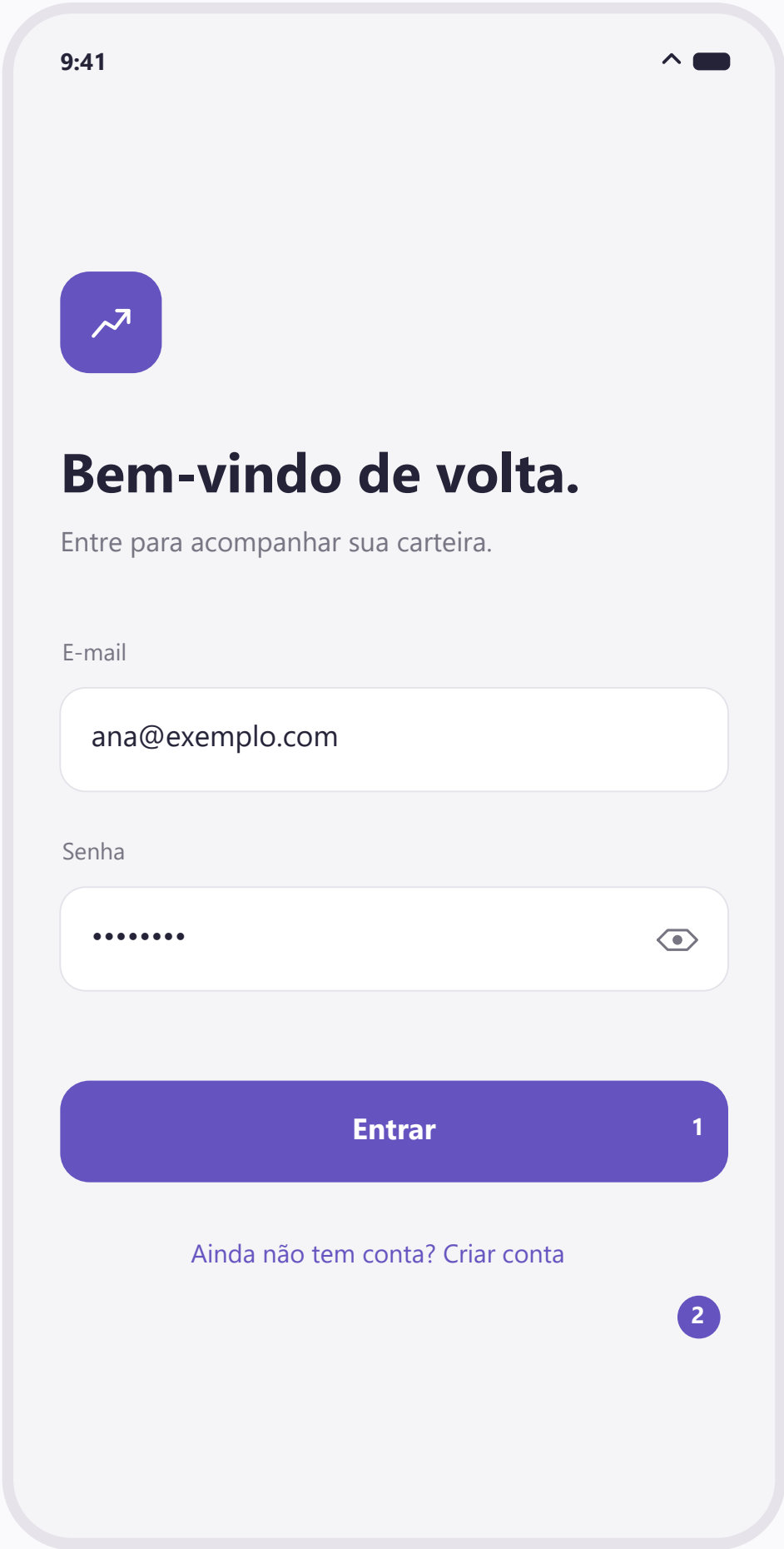




Entrar

TELA E INTEGRAÇÕES

1 Botão Entrar

```
Local · database.login(email, password)
```

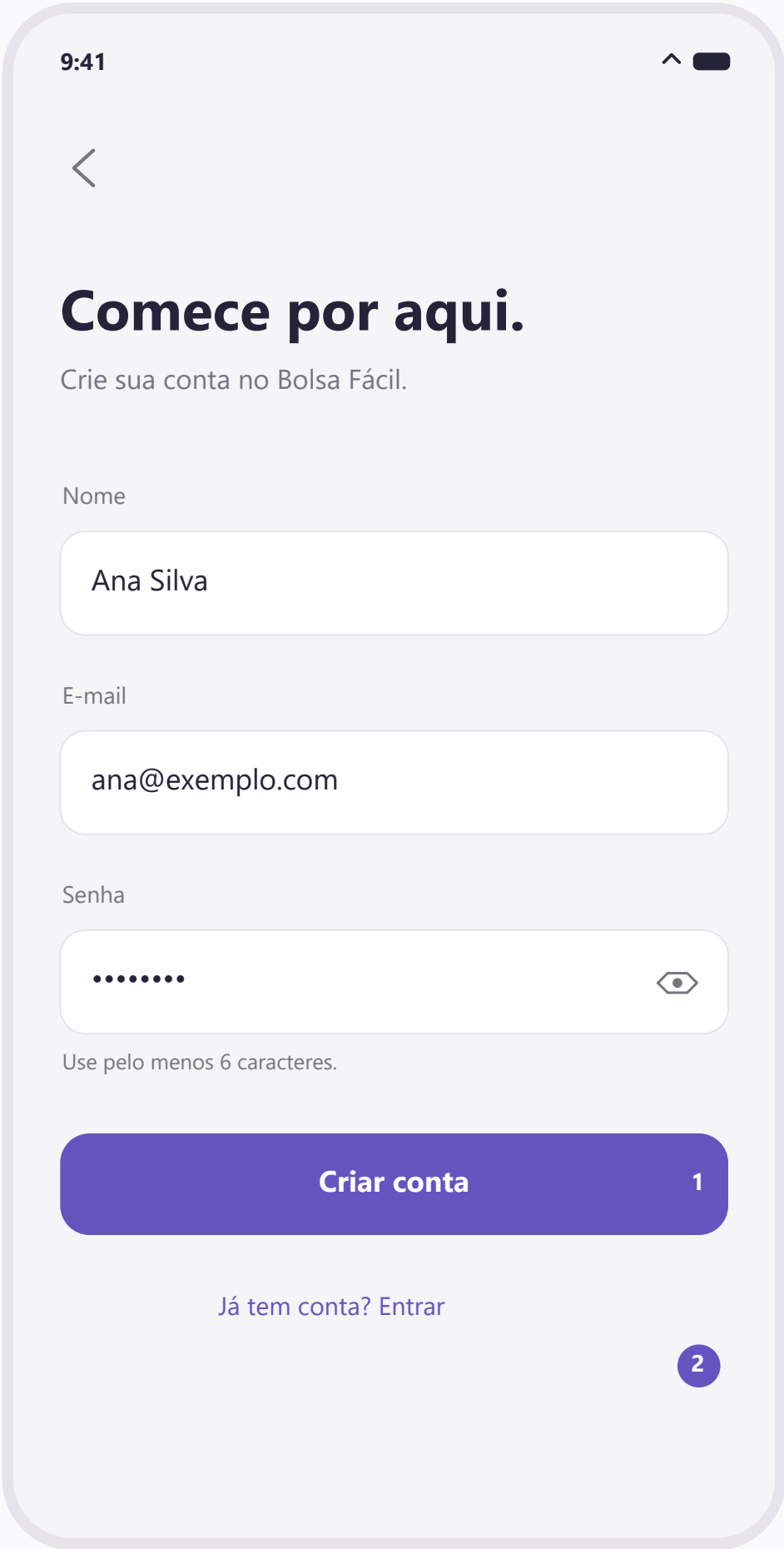
Valida as credenciais no SQLite e salva a sessão neste dispositivo. Durante o envio, o botão mostra progresso; erros aparecem no formulário.

2 Após entrar: carregar o mercado

```
GET /quote/{ticker}
```

Após autenticação local, refresh() consulta as cotações de cada ativo e carrega favoritas e posições. A lista é exibida no Início.

Sem endpoint HTTP de login. Bases GET: Web localhost:8081/api; nativo brapi.dev/api. O marcador 2 indica o próximo passo do fluxo.



Criar conta

TELA E INTEGRAÇÕES

1 Botão Criar conta

```
Local · database.register(name, email, password)
```

Valida os dados e cria a conta e a sessão no armazenamento local. Campos inválidos recebem uma mensagem no formulário.

2 Após criar: carregar o mercado

```
GET /quote/{ticker}
```

Executa refresh() para abrir o Início com cotações. Nome, e-mail e senha não são enviados à brapi.

Conta local ao dispositivo. Não existe endpoint HTTP de cadastro. O marcador 2 indica o próximo passo do fluxo.



Início

TELA E INTEGRAÇÕES

1 Campo de busca

```
GET /quote/{ticker}?range=3mo&interval=1d
```

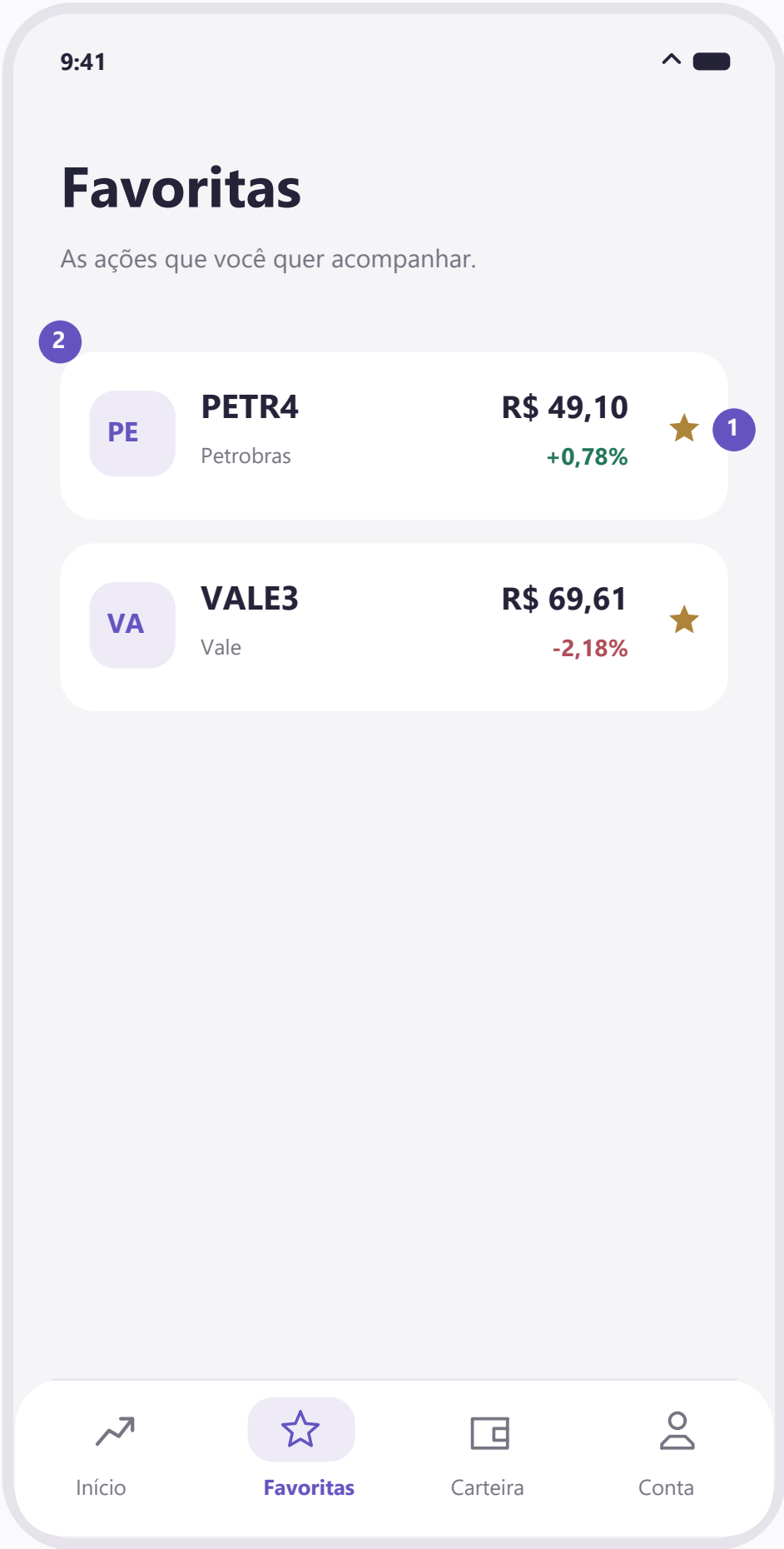
Consulta uma ação fora do cache. Retorna nome, preço, variação e histórico. Normaliza o ticker para maiúsculas. Se já estiver no cache, a busca usa os dados em memória.

2 Lista de ações e atualização

```
GET /quote/{ticker}
```

Uma consulta por ticker. Preenche os cards e atualiza as cotações ao puxar a lista. Reúne ações padrão, favoritas e posições, sem duplicatas. A estrela grava localmente, como na tela Favoritas.

Removido o painel Dólar / Selic / IPCA, sem integração no código atual. Falha de carga: mensagem e Tentar novamente.



Favoritas

TELA E INTEGRAÇÕES

1 Estrela de cada ação

```
Local · database.setFavorite(userId, symbol, enabled)
```

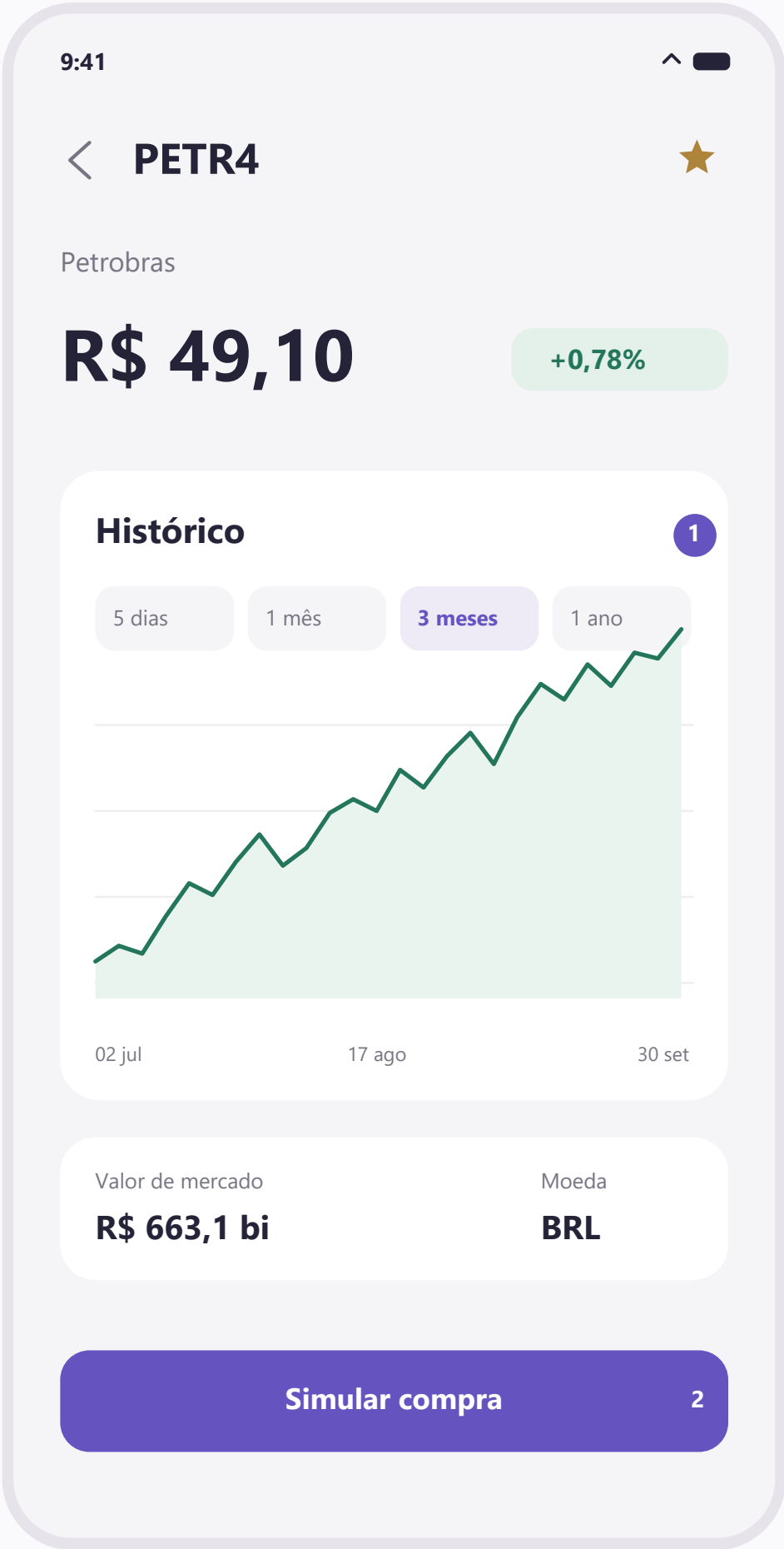
Adiciona ou remove o ticker da conta. A lista é lida por favoritesFor(userId). A estrela não faz uma requisição HTTP.

2 Preços dos cards: puxar para atualizar

```
GET /quote/{ticker}
```

Atualiza cotações via refresh(). Abrir a aba usa o cache existente; tocar em uma ação abre detalhes e consulta o histórico.

Lista vazia: "Nenhuma favorita ainda". Removido o card explicativo permanente.



Detalhes da ação

TELA E INTEGRAÇÕES

1 Cotação, gráfico e valor de mercado

GET /quote/{ticker}?range=3mo&interval=1d

Consultado ao abrir a ação. Os períodos usam range=5d, 1mo, 3mo ou 1y, com interval=1d. historicalDataPrice fornece datas e fechamentos; marketCap preenche o valor de mercado.

2 Botão Simular compra

Navegação · abrir área de compra

Abre quantidade e total da mesma ação, na próxima página. É uma proposta de composição: a compra atual fica no conteúdo rolável de detalhes. A estrela usa setFavorite localmente.

Removidos P/L, P/VP, dívida, caixa e dividendos: os blocos das prints não são consumidos pelo modelo atual.



Compra simulada

TELA E INTEGRAÇÕES

1 Preço da ação e cálculo do total

```
GET /quote/{ticker}?range=3mo&interval=1d
```

O preço vem da consulta feita em detalhes: regularMarketPrice. Alterar a quantidade recalcula quantidade × cotação; não consulta um novo endpoint.

2 Botão Adicionar à carteira

```
Local · AppState.buy(symbol, quantity, price)
```

Salva a posição pelo preço exibido. Se já existir, soma as ações e recalcula o preço médio ponderado. Usa savePosition no SQLite. Quantidade inválida desativa o botão; sucesso exibe confirmação.

Não há endpoint de ordens. A área separada simplifica a apresentação; não altera o código do app.



Carteira

TELA E INTEGRAÇÕES

1 Resumo e valores de cada posição

Cache · cotações de GET /quote/{ticker}

Lê positionsFor(userId) no SQLite. O valor atual usa quantidade × cotação em cache; sem cotação, usa preço médio. Abrir a aba não dispara HTTP. Resultado = valor atual - investido.

2 Menu da posição: editar ou remover

Local · savePosition / removePosition

Altera quantidade e preço médio ou remove a posição. Se o ticker salvo estiver ausente do cache, refresh() consulta GET /quote/{ticker} para o conjunto completo.

Toque no ativo para detalhes; use o menu para editar. Adicionar abre o formulário de nova posição e usa savePosition.



Editar posição

TELA E INTEGRAÇÕES

1 Botão Salvar alterações

```
Local · database.savePosition(userId, item)
```

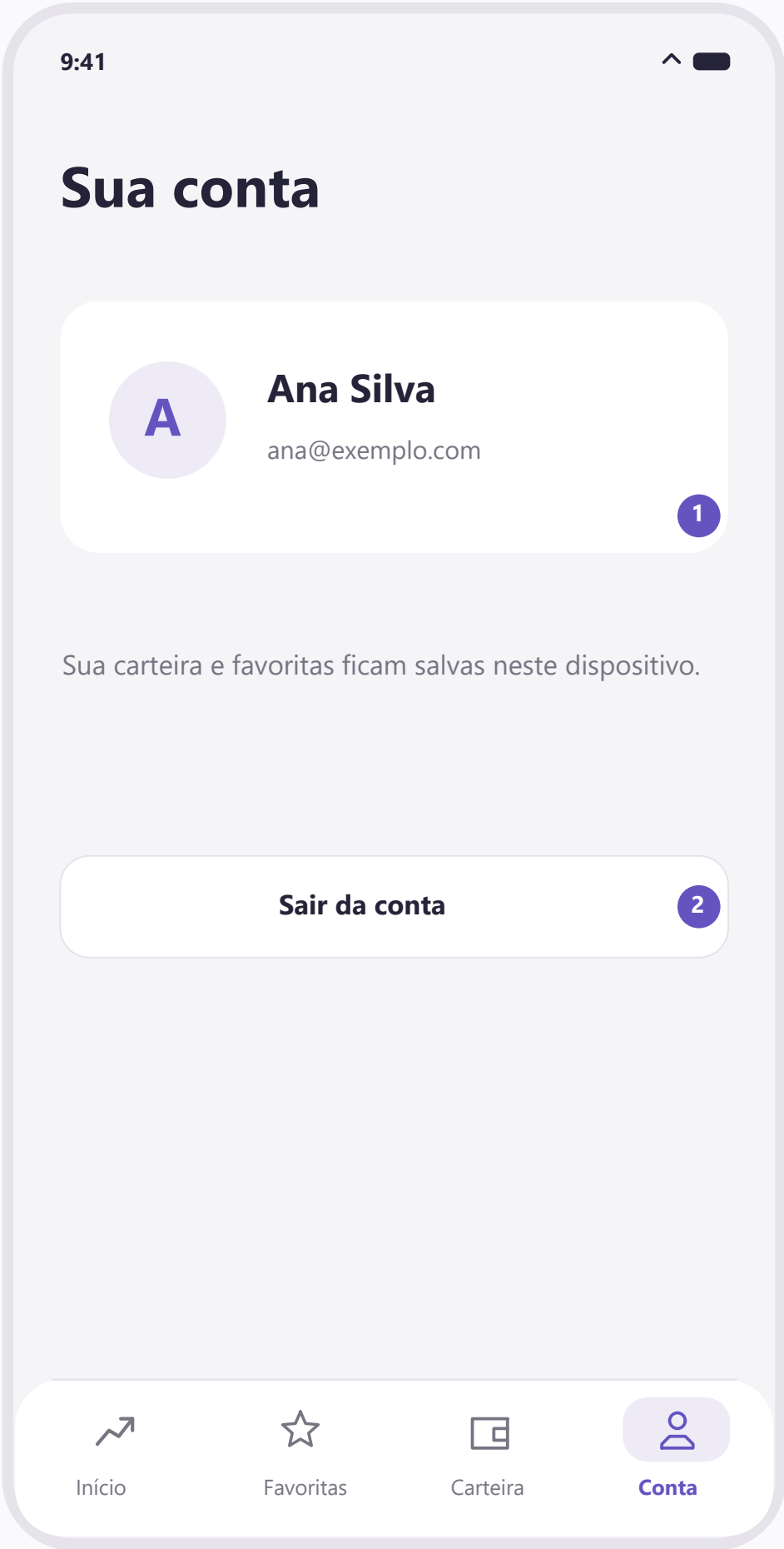
Grava quantidade e preço médio na conta. Ambos devem ser maiores que zero. O ticker fica fixo na edição. Para adicionar uma posição, o formulário permite informar o ticker.

2 Remover posição

```
Local · database.removePosition(userId, symbol)
```

Remove a posição da carteira local. Não envia ordem de venda nem faz uma consulta HTTP.

Novo ticker fora do cache: salvar dispara refresh(), usando GET /quote/{ticker}. A edição usa os dados carregados.



Conta

TELA E INTEGRAÇÕES

1 Nome e e-mail

```
Local · restoreSession() / currentUser
```

O perfil vem da conta local restaurada na inicialização. Carteira e favoritas são separadas por usuário. Não há endpoint de perfil.

2 Botão Sair da conta

```
Local · database.logout()
```

Remove a sessão e limpa o estado em memória, retornando ao login. Não existe endpoint HTTP de logout.

Bases GET: Web <http://localhost:8081/api>; nativo <https://brapi.dev/api>. BRAPI_BASE_URL substitui a base. No Web, o proxy adiciona a credencial.